Trusted Execution Environments: What They Are, Associated Challenges, and Future Directions for Improvement

ETHAN O'CONNOR, University of Massachusetts - Amherst, United States of America

Trusted Execution Environments (TEEs) use hardware to isolate code and data from the operating system (OS) and processes running on a machine. This creates a secure area where processes and applications can keep their confidential data. TEEs protect that data while it is actively being used, as that is when encryption cannot protect the data. This report explains TEEs: their definition, the reason they were created, and how they are currently being used. The examples shown are real TEEs used today, like Intel SGX, AMD SEV-SNP, and Arm TrustZone to name a few. Building off of research papers and surveys, this paper then analyzes five challenges related to TEEs: side-channel leaks, incomplete attestation, limited resources while handling a growing trusted computing base, passing trust to GPUs and other I/O, and how to determine trust in the code running inside the TEE. This paper also explains five directions that could improve current implementations of TEEs: a compound attestation setup, side-channel cognizant design, using verified TEE firmware, confidential AI at large scales, and portable/open TEE systems.

1 Introduction

1.1 Definition

A Trusted Execution Environment (TEE) is a boundary created by hardware, where the code and data held within is handled with integrity and confidentiality guarantees. TEEs are designed so if high privilege software like the OS, hypervisor, or firmware is compromised, the data inside the TEE is still safe. The term was first defined in the "OMTP Advanced Trusted Environment: OMTP TR1" specification, initially released in 2008, later revised in 2009 [1]. Eventually, GlobalPlatform later published its own TEE definition [5]. There are two abilities that define modern TEEs: The first is its ability to isolate code and data. This means that the CPU prevents access to protected memory areas, and can encrypt the memory in DRAM. This makes it so untrusted, but privileged software cannot compromise or read the data. The second ability is remote attestation: the TEE creates some cryptographic evidence, for example a hash of the loaded TEE code, so one party can check that the other party is not compromised before sending secrets [6, 9]. Using the Bob and Alice analogy, this allows Bob to use a verifying party (which may be himself) to ensure that Alice's TEE has not been tampered with maliciously.

The secure computing industry has closed in on a few main design types for TEEs, which Forough et al. grouped into four design points; two of the most common are described here [6]. The first type is process-level enclaves, such as Intel SGX, protect aspects of applications while keeping the trusted computing base small. VM-level designs, such as AMD SEV-SNP and Arm CCA, protect entire virtual machines, and let software run inside them [6, 9]. Table 1 summarizes a few of the main platforms used currently.

Author's Contact Information: Ethan O'Connor, eroconnor@umass.edu, University of Massachusetts - Amherst, Amherst, MA, United States of America.

Table 1. TEE platforms, summarized from [6–9].

Platform	Isolation unit/type	Notable attributes
Intel SGX	Process enclave	Small TCB; limited enclave memory; code must be partitioned; built-in remote attestation
Intel TDX	Trust domain (VM)	VM isolation; extends SGX-like attestation
AMD SEV-SNP	Virtual machine	Runs legacy software; larger trusted computing base; attestation at launch and at runtime
Arm TrustZone	Secure world	Present on mobile and embedded devices; a static secure world; no built-in attestation
Arm CCA	Dynamic VM	Realms managed by untrusted software; firmware security verified
NVIDIA H100 CC	GPU confidential mode	Encrypted CPU to GPU link; relies on a CPU TEE

1.2 Motivation

Secure computing relies on the fact that the platform under a process is trustworthy. In the case of cloud computing, this assumption is difficult to prove true: the user must trust the provider's hypervisor, admins, and general hardware/software stack, which can have many vulnerabilities present. Software defenses can be circumvented by a malicious actor with enough privileges, and cryptographic alternatives like fully homomorphic encryption and secure multi-party computation can increase the workload of a task by two to four orders of magnitude [6]. TEEs present a reasonable alternative, as they disassociate system software (like the OS) from the trusted computing base, while still allowing code that needs to process quickly, to run at close to native speeds. Surveys in this industry, such as an IDC study found that 75% of organizations were adopting confidential computing, with 18% already in production [4].

1.3 Applications

TEEs are typically used when sensitive data is processed on devices which the owner of the data does not control. Some examples of this are:

- Cloud services and cloud databases: SGX-based systems such as VC3 and EnclaveDB run analytics and SQL queries over encrypted data. Library operating systems such as SCONE and Graphene-SGX transfer unmodified applications into secure enclaves [6].
- Machine learning (ML) and large language model (LLM) inference: Model weights and user prompts stay protected/encrypted

from the cloud operator. The Azure confidential GPU platform, built on NVIDIA GPUs, is an example of one deployment [7].

- Mobile and internet of things (IoT) security: TrustZone protects security sensitive code and data, like one time password tokens, on mobile platforms including phones and tablets [6, 11], while TrustZone-M brings isolation to resource-limited IoT devices [9].
- Agentic Artificial Intelligence (AI): Forough et al. surveyed the use of TEEs to protect LLM-driven agents against a compromised cloud operator. These agents hold confidential credentials and collect sensitive context information [6].
- Edge ML: DICE-anchored realms on the Arm CCA platform were proposed to protect model weights and prompts on edge devices [2]. This protection would prevent malicious actors from abusing models, for example by extracting their weights or tampering with their guardrails [2].

The rest of this report looks at five challenges TEEs face currently (Section 2), and five directions that the industry could go in to improve TEEs for better secure computing in the future (Section 3).

2 Technical Challenges

2.1 The First Challenge: Architectural Leaks and Side-Channel Issues

TEEs are designed to protect against direct reads of secure memory by privileged software, but they leave many other indirect channels open for exploitation. Fei et al. surveyed the vulnerabilities of SGX and looked at possible countermeasures. They concluded that side-channel attacks remained a significant threat to SGX, as many of the published attacks were based on side-channel leaks [5]. Because an untrusted OS manages the enclave's memory, page-fault patterns and interrupt timings can be inspected and used in side-channel attacks. It can also share caches and other architecture resources with the enclave, which leaves the secure enclave vulnerable. Puddu's study of privileged attackers displayed this issue, frequent interrupts from the OS could expose detailed instruction timings, causing encryption keys to be leaked from the TEE [11].

This issue is not exclusive to the SGX design, though. For AMD SEV, Wichelmann et al. explained a ciphertext side channel attack. The SEV encrypts memory in a deterministic fashion for a given memory address, and if a hypervisor watches the ciphertext changes, it can learn information about the plaintext. Code written to be run in constant-time and other hardware fixes did not mitigate this issue completely, but the Cipherfix framework they presented was shown to protect against these issues [17]. On top of this, agentic AI workloads add more concerns. Autoregressive token generation has predictable memory access patterns, and Forough et al. noted that side-channel analysis for attention, KV-cache access, and speculative decoding have not been well-studied in TEEs [6]. Another difficulty is that many vendors handle side channel vulnerabilities as outside the scope of their threat model. This puts the burden of mitigating these issues onto developers, who would need tools to measure these leaks in realistic applications [3].

2.2 The Second Challenge: Incomplete and Hard-to-Compose Attestation

The basis of trust in remote computing models is heavily reliant on attestation, but the main TEE platforms handle it in different fashions. Ménétrey et al. found that the SGX platform offered a fully built-in remote attestation protocol, intended for multiple separate applications. Arm TrustZone and the native RISC-V architecture provide no way to certify software in the trusted execution environment, which has forced the community to propose improvised methods [9]. Attestation implementations have also been demonstrated to be flawed in their design. For example, the initial AMD SEV attestation protocol was shown to be unsafe, causing systems to be exposed to rollback attacks, and it also allowed a malicious actor with physical access to install custom firmware [9]. When an implementation depends on a vendor-operated service, like Intel's attestation service for EPID, this creates another external dependency. An alternative, DCAP, allows verifiers to run their own infrastructure [9].

This means that when attestation actually works, it only answers a specific question, with that being "Is the expected code running on actual TEE hardware?", not that there are no issues with the configuration. Forough et al. noted exactly this, attestation proved that certain code was loaded on a TEE platform, but it does not prove that the code does not have other malicious attacks present, like a supply-chain attack. Attestation also does not by itself prevent rollback or replay, where a secure enclave or confidential VM is reverted to an older snapshot or fed stale attestation evidence [6]. Current TEE protocols are designed for "one-to-one" verification, where a single verifier can attest to a single enclave. In the real world, systems are more complex than that. Multiple-part processes, like logging in through Google on a website, have many parts that could run on different TEEs and hardware. There are no TEE platforms that currently support multi-attestation checking across these processes, out of the box [6].

2.3 The Third Challenge: Limited Resources in a Growing TCB

All of the current TEE designs have major trade offs: those being TCB size, performance, and ease of programming. For example, the SGX implementation keeps the TCB small, but Ménétrey et al. found that the enclave's page cache is limited to around 93.5 MB on the majority of systems, and if this limit was exceeded, the memory-access latency increased greatly due to how paging works. Current server implementations raise that limit to 512 GB, but the integrity protection of the data is dropped, decreasing protection against physical attacks. The secure enclave model also prevents direct hardware access and system calls, but then processes that need the enclave are forced to be split into trusted and untrusted parts [9]. For Arm TEEs, the open source 'OP-TEE trusted OS' forces a memory limit to a few megabytes and has no POSIX API available, which causes the process of porting legacy code to be difficult [9].

For VM-based TEEs, they mostly eliminate the issue of porting, since the original environment is replicated. The major tradeoff is that they have a large TCB, because the entire environment (OS and application included) is in the trusted computing base [9]. Forough et al. also stated that the Intel TDX platform offered weaker internal

separation of trusted code, once that code has entered the trusted base. This is not ideal as, even if a program is trusted, it could get hijacked and provide other program's TCB data to a malicious actor [6]. It was also stated that SEV-SNP's individual VM encryption keys stopped the sharing of protected memory between coinciding (or side-by-side) VMs, but that this hurt designs that share the model state between each other [6]. This forces real-world implementers to choose between specifically tuned secure enclaves, and convenient but large confidential VMs.

2.4 The Fourth Challenge: Passing Trust to GPUs and I/O

CPU TEEs are not able to protect data that has to move onto a GPU, like in the case of machine learning. Volos et al. showed Graviton, a method that allowed a GPU to have TEE capabilities with minimal hardware complexity and low performance impact (with 17 to 33% overhead). They addressed attack vectors like a malicious OS tampering with GPU commands/data, and isolation / memory access violations [16]. Support for commercial applications began when NVIDIA introduced the Hopper architecture. Gu et al. analyzed this design, where the GPU could (from boot) start into a secure state, with the CPU to GPU link being encrypted, and where the GPU could not operate on its own. but had to work together with a CPU TEE [7]. Forough et al. mentions that this does not, by itself, create end to end separation: the GPU depends on a trusted CPU TEE, so the GPU's setup path through drivers, direct memory access (DMA) paths, and device-scheduling metadata remain as potential attack vectors [6].

The performance hit onto systems using this setup is also an issue. Chrapek et al. tested H100 GPUs in confidential mode for LLM inference and had a tested throughput overhead of 4% to 8%, which decreases as batch and inputs grow in size [19]. Zhu et al. reported an overhead below 7% for usual queries on 8B to 70B parameter models, with larger models having almost no additional overhead [18]. Both studies used a single GPU, and single model inference, so these results may not be transferrable to multi-agent workloads that share GPU resources [6]. There are also issues on other platforms, like in the case of integrated devices. For example, Devlore expands on the Arm CCA platform, protecting realm VMs that use integrated peripherals from interrupt manipulation attacks, without changes to applications or device drivers [15].

2.5 The Fifth Challenge: Trusting Code Running Inside the TEE

A TEE only can guarantee that the code/data inside it runs in isolation. It does not report if the code is secure, that is, if an exploit was recently developed against a TEE-using program, it would not prevent that vulnerable code from running. Ning et al. stated that the code running inside a TEE directly effects the security of the whole TEE: a vulnerability present in the code inside the TEE could be abused by a malicious actor, which is then used to corrupt the TEE itself. The paper also noted that the field lacks the ability to verify code executing on the TEE. It also lacks defenses inside the trusted environment, it lacks methods to detect compromised environments, and it lacks any plan to mitigate an attack when it occurs [10].

A response to this issue is having formal verification. Li et al. designed the Arm CCA where the functionality lives on a small firmware component, which they named the Realm Management Monitor, and used Coq to verify its C and assembly implementation. They proved that this design creates a system equal to an ideal secure machine, which was the first proof of security guarantees in a confidential computing architecture [8]. Their design makes the complexity shift from hardware to firmware, making verification a critical component. Comparing to currently used TEEs, they have no similar proofs [8], although formal analysis of the SGX existed, certain aspects of SGX such as its secure enclave and microcode are closed source [9]. Closed source models shut down attackers from nitpicking code for the first day, maybe week, but closed-source components prevent independent verification of security claims. Security through obscurity is not security. Forough et al. also explained how performance shortcuts can negatively effect protection capabilities. A reused key flaw in an optimization for partly protected inference allows an attacker recover model weights [6].

3 Possibilities for Improvements in the Future

3.1 Compounding Attestation, and the Standardization of Evidence

A design that could increase TEE security is an attestation platform or architecture for multi-process TEEs. In the Google login example, if there was a system that could follow and verify the chain of TEEs needed to back up that process, this would increase the security of the process as a whole. This would package many aspects of verification: Execution integrity would allow for checking of, "did the code run correctly?". Provenance would allow a piece of code or data inside the TEE to have a known source. Revocation would give the ability to a TEE to cancel trust after code or data was inserted inside the TEE. Scope checking would allow the TEE to see if a program broke out of its intended bounds. Efficient verification of this "chain of checks" would be ideal, if each sub-check can be verified quickly, it would not impact the performance greatly. Forough et al. stated that no complete solution had been proposed for this form of verification, although blockchain-based approaches partially addressed the issue [6]. A related project is the IETF RATS, which provides the vocabulary of attesters, verifiers, and relying parties [9]. Another related project is DICE-anchored realms, which link Arm CCA's attestation tokens to a device ID framework [2]. In certain applications, like high risk AI systems or financial infrastructure, deployment may require to show regulators machine-readable evidence that show which model, runtime, and hardware processed protected data [6].

3.2 Side-Channel Cognizant Hardware, Software, and Tools

In the future, TEEs need the ability to resist side-channel attacks. This should be treated as an urgent design goal rather than an afterthought or an idea that is out of scope. As stated previously, work covering this ability includes software defenses such as Cipherfix [17]. There are also detection based defenses that provide secure enclaves a trusted source of microarchitectural information, like

SENSE [12]. Leak measuring platforms (or updates to current platforms) for confidential VMs would allow a tester to estimate what an attacker would learn from something like a side-channel attack, could prove useful to improve security [3]. In the case of AI security, an in-depth study covering leaks from attention patterns, KV-cache accesses, and speculative decoding has not been published yet as of Forough et al.'s survey [6], and a survey like it may greatly improve the security of models.

3.3 Verification of TEE Firmware AND Hardware

The CCA verification model showed that proofs for TEE security monitors are possible, although what was verified was an early prototype of the CCA firmware [8]. The question is, how can this proof be expanded to other platforms? Or to systems that lack TEE-secure memory, to all the devices used in an attestation chain (like in the Google login example), to device assignment for a virtual machine, and to verification of GPUs or other I/O devices? Open-source monitors, like Sanctum and Keystone for the RISC-V architecture, replaced hidden/opaque parts of a TEE, like Intel's closed source microcode, with open source components, which can point towards more transparent roots of trust (which may be required) [9]. Ning et al. also stated that platforms that can hunt for vulnerabilities in TEE-using code may be necessary, alongside the ability to detect compromised TEEs at runtime [10].

3.4 Confidential AI and Agentic Systems at Scale

The need for confidential computing in AI setups is growing rapidly, but Forough et al. concluded that no universally accepted end to end framework links the hardware into a single, easy to use system for agents. Some of the current problems include attested vector stores for retrieval, attestation cognizant agent setups and protocols, and GPU TEE performance at a large language model scale [6]. There is some work in the field like EnclaveX, which allows for the combination of CPU and GPU TEEs, while trying to remove the Kubernetes admin from the trusted computing base [13]. Their work pushes towards this end to end framework idea, but it is not industry standard yet. It is also important to note, TEEs do not solve other LLM issues, such as prompt injection attacks and model supply chain risks, so TEEs must be combined with software and application defenses [6].

3.5 Portable/Open, and Heterogeneous TEE Systems

Ménétrey et al. observed a convergence toward VM-based isolation, and they argue that it may ease application development, since the OS and APIs inside the VM are standardized. They also pointed to hardware-agnostic projects like the Open Enclave SDK, and WebAssembly runtimes, which could "put a wall" or hide attestation behind standard interfaces, making applications portable across TEEs [9]. In a survey of RISC-V enclaves, it was identified that there are still challenges in moving toward an all in one trusted computing system [14]. By making applications and processes portable across the TEEs, like SGX, SEV-SNP, TDX, and CCA, it may be beneficial for the industry, especially if it is found that it is possible to keep all security properties.

4 Conclusion

TEEs have grown from their initial specification, as outlined by OMTP initially in 2008, revised in 2009 [1], into a crucial aspect of cloud, mobile, and AI security infrastructure. The literature reviewed in this paper agrees on the great value provided by these platforms, but they also agree that many of them are limited in their abilities to protect against complex or fine tuned attacks. This means that the platforms may defend against a simple, privileged adversary, but remain highly vulnerable to other attacks like side-channel based ones. They also depend heavily on varying attestation protocols, they force trade-offs between TCB size and general usability of the TEE platform, need to be adaptated to support GPUs, and have problems with checking the trust level of the code and data placed inside TEEs. The ideas presented in Section 3 are outlines of ideas presented across these papers and studies, but they are not ready to implement solutions.

Acknowledgments

This report was prepared as a literature-based study. Unless otherwise noted, claims are taken from the papers cited.

Ethics and Privacy Statement

This report is only a review of pre-existing literature. It does not involve any new studies or datasets, and therefore ethics and privacy are not a major issue.

References

[1] Open Mobile Terminal Platform (OMTP). 2009. Advanced Trusted Environment: OMTP TR1, v1.1. First released in 2008, revised in 2009. https://www.gsma.com/newsroom/gsma_resources/omtp-documents-1-1-omtp-advanced-trusted-environment-omtp-tr1-v1-1/

[2] Lorenzo Ferro, Silvia Sisinni, Flavio Ciravegna, Enrico Bravi, and Antonio Lioy. 2026. DICE-anchored Realms on Arm CCA. In the Joint National Conference on Cybersecurity (ITASEC & SERICS 2026). Cagliari, Italy. https://ceur-ws.org/Vol-4198/paper40.pdf

[3] Ruiyi Zhang, Albert Cheu, Adria Gascon, Daniel Moghimi, Phillipp Schoppmann, Michael Schwarz, and Octavian Suciu. 2025. FARFETCH'D: A Side-Channel Analysis Framework for Privacy Applications on Confidential Virtual Machines. https://doi.org/10.48550/arXiv.2506.15924.

[4] International Data Corporation (IDC). 2025. Unlocking the Future of Data Security: Confidential Computing as a Strategic Imperative. IDC White Paper by Philip Bues, Document Number #US53866125, sponsored by the Confidential Computing Consortium. https://confidentialcomputing.io/wp-content/uploads/sites/10/2025/11/US53866125.pdf

[5] Shufan Fei, Zheng Yan, Wenxiu Ding, and Haomeng Xie. 2021. Security Vulnerabilities of SGX and Countermeasures: A Survey. ACM Computing Surveys Volume 54, Issue 6 (2021). https://doi.org/10.1145/3456631

[6] Javad Forough, Marios Kogias, and Hamed Haddadi. 2026. When Agents Handle Secrets: A Survey of Confidential Computing for Agentic AI. https://doi.org/10.48550/arXiv.2605.03213

[7] Zhongshu Gu, Enriquillo Valdez, Salman Ahmed, Julian James Stephen, Michael Le, Hani Jamjoom, Shixuan Zhao, and Zhiqiang Lin. 2025. Blueprint, Bootstrap, and Bridge: A Security Look at NVIDIA GPU Confidential Computing. https://doi.org/10.48550/arXiv.2507.02770

[8] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. 2022. Design and Verification of the Arm Confidential Compute Architecture. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). https://www.usenix.org/system/files/osdi22-li.pdf

[9] Jämes Ménétrey, Christian Göttel, Anum Khurshid, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, and Shahid Raza. 2022. Attestation Mechanisms for Trusted Execution Environments Demystified. In the 22nd International Conference on Distributed Applications and Interoperable Systems (DAIS 2022). Springer. https://doi.org/10.1007/978-3-031-16092-9_7

[10] Zhenyu Ning, Fengwei Zhang, Weisong Shi, and Weidong Shi. 2017. Position Paper: Challenges Towards Securing Hardware-assisted Execution Environments.

In the Proceedings of the Hardware and Architectural Support for Security and Privacy (HASP 2017). ACM. https://doi.org/10.1145/3092627.3092633

[11] Ivan Puddu. 2023. Understanding the Capabilities of Privileged Attackers Against Trusted Execution Environments. Doctoral thesis at ETH Zurich, https://doi.org/10.3929/ethz-b-000610177

[12] Fan Sang, Jaehyuk Lee, Xiaokuan Zhang, Meng Xu, Scott Constable, Yuan Xiao, Michael Steiner, Mona Vij, and Taesoo Kim. 2024. SENSE: Enhancing Microarchitectural Awareness for TEEs via Subscription-Based Notification. In the Network and Distributed System Security Symposium (NDSS 2024). https://www.ndss-symposium.org/ndss-paper/sense-enhancing-microarchitectural-awareness-for-tees-via-subscription-based-notification/

[13] Robert Schambach, Quoc Do Le, Sergei Arnautov, and Christof Fetzer. 2026. EnclaveX: End-to-End Confidential AI with CPU/GPU TEEs. Preprint, https://doi.org/10.48550/arXiv.2606.31408.

[14] Marouene Boubakri and Belhassen Zouari. 2025. A Survey of RISC-V Secure Enclaves and Trusted Execution Environments. https://doi.org/10.3390/electronics14214171

[15] Andrin Bertschi, Supraja Sridhara, Mark Kuhne, Benedict Schlüter, Friederike Groschupp, Clément Thorens, Nicolas Dutly, Srdjan Capkun, and Shweta Shinde. 2024. Devlore: Device Interrupt Protection for Confidential VMs. https://doi.org/10.48550/arXiv.2408.05835

[16] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted Execution Environments on GPUs. https://www.usenix.org/system/files/osdi18-volos.pdf

[17] Jan Wichelmann, Anna Pätschke, Luca Wilke, and Thomas Eisenbarth. 2022, revised 2023. Cipherfix: Mitigating Ciphertext Side-Channel Attacks in Software. https://doi.org/10.48550/arXiv.2210.13124

[18] Jianwei Zhu, Hang Yin, Peng Deng, Aline Almeida, and Shunfan Zhou. 2024. Confidential Computing on NVIDIA Hopper GPUs: A Performance Benchmark Study. https://doi.org/10.48550/arXiv.2409.03992

[19] Marcin Chrapek, Marcin Copik, Etienne Mettaz, and Torsten Hoefler. 2025. Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs. https://doi.org/10.48550/arXiv.2509.18886